

Heat Transfer Coefficient Surrogate Model

Overview

A first-principles heat transfer coefficient is determined for a cylinder in cross flow at specific flow velocities. The calculated heat transfer coefficient is then used to anchor a computational fluid dynamics (CFD) model in Ansys Fluent. Following the anchoring of the model, three more configurations of the cylinder in cross flow are simulated in Ansys Fluent, including two cylinders in parallel, two cylinders in series, and a matrix of four cylinders. The simulations are then parameterized and ran at a batch level to develop 456 data points. A surrogate model is developed using Machine Learning techniques to apply discretized features of the data from the simulations. The goal of the surrogate model is to provide predictive insight without the need for developing additional simulations in Ansys Fluent.

First-Principles Anchoring

Bergman, et. al., outlines a correlation to determine the heat transfer correlation for a cylinder in cross flow. [1] Specifically, Equation 7.52 outlines the following correlation, relating the Nusselt, Reynolds, and Prandtl numbers.

$$\bar{Nu} = CRe_D^m Pr^{1/3}$$

The Reynolds number can be determined using the following expression [1]:

$$Re = \frac{\rho v D}{\mu}$$

Where the numerator consists of the density and velocity of the fluid, and the characteristic diameter of the object. The denominator consists of the

Since the Nusselt number is the ratio of the convective and conductive heat transfer, it is also equivalent to the following expression [1]:

$$\bar{Nu} = \frac{\bar{h} D}{k}$$

Setting the two expression for the Nusselt number equivalent to one another and solving for the heat transfer coefficient provides the following expression [1]:

$$\bar{h} = \left(\frac{k}{D} \right) CRe_D^m Pr^{1/3}$$

From Bergman, et. al., the values for C and m in the equation above are dependent on the Reynolds number as depicted in the table below [1].

Re	C	m
0.4-4	0.989	0.330
4-40	0.911	0.385
40-4,000	0.683	0.466

Re	C	m
4,000-40,000	0.193	0.618
40,000-400,000	0.027	0.805

Three arbitrary velocities will be selected and their respective Reynolds Number and Heat Transfer Coefficient are calculated.

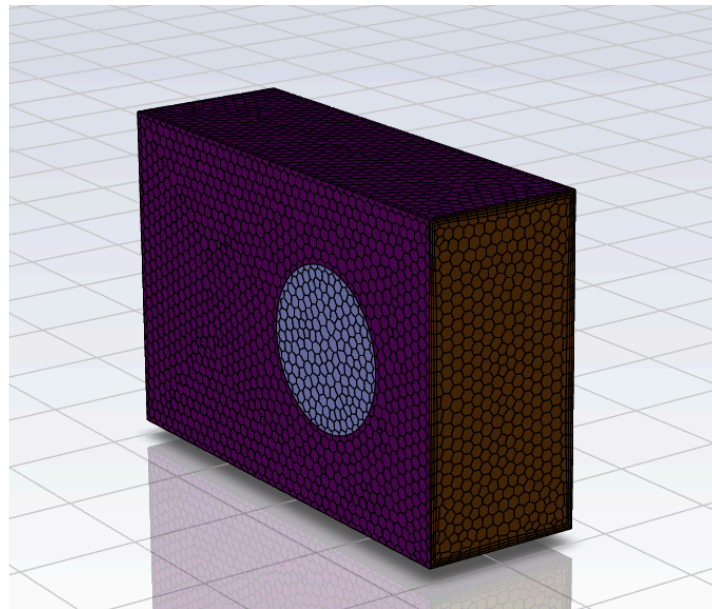
Velocity (m/s)	Re	H ($\frac{W}{m^2K}$)
3	4794	33.51
5	7990	45.95
7.5	11985	59.03

*Please refer to the appendix for the Python script used for the Reynolds number and heat transfer coefficient calculations in the table above.

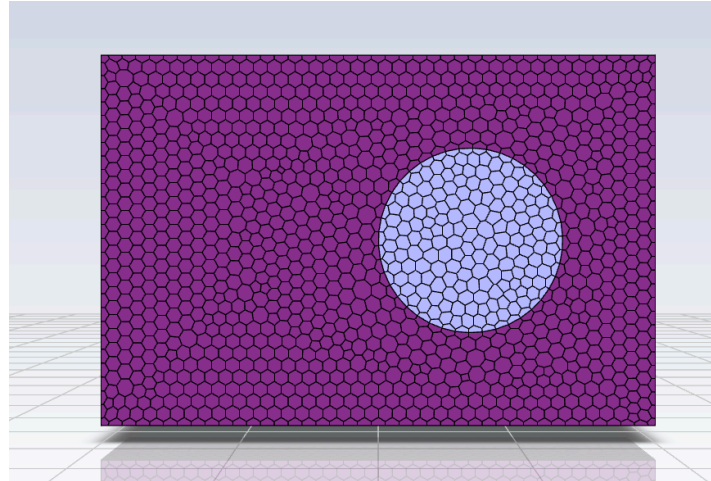
These will be used as the anchoring values for the single cylinder CFD analysis in Ansys Fluent.

Single Cylinder - Ansys Fluent

The geometries used for the CFD models are 3-dimensional because it demonstrates added complexity and allows for a simpler correlation to the first-principles heat transfer correlation.



In the visual below, the darker portion of the mesh models the air flowing across the cylinder. The lighter portion of the mesh models the solid cylinder, where the heat transfer of interest occurs. The opening on the left and right are modeled as a velocity inlet and pressure outlet, respectively.



K-Omega is used due to the number of near-wall behaviors of the flow. The K-Omega is also anticipated to be more precise, and won't require the addition of a wall function. To perform batch simulations, wall functions are avoided to prevent manual reconfiguring across the models. The energy modeled is used to calculate the heat transfer coefficient.

The mesh is compliant with the following metrics for every simulation.

Metric	Skewness	Quality	Aspect Ratio
Value	<0.8	>0.2	10

The values for the heat transfer coefficient from Ansys Fluent can be found in the table below.

Velocity ($\frac{m}{s}$)	CFD H-Value ($\frac{W}{m^2K}$)
3	31.42
5	46.26
7.5	63.28

The heat transfer coefficient values below will be compared to the first principle results calculated earlier to anchor the CFD analysis.

Velocity (m/s)	First Principle H-Value ($\frac{W}{m^2K}$)	CFD H-Value ($\frac{W}{m^2K}$)	Error (%)
3	33.51	31.42	6.65
5	45.95	46.26	-0.67
7.5	59.03	63.28	-6.72

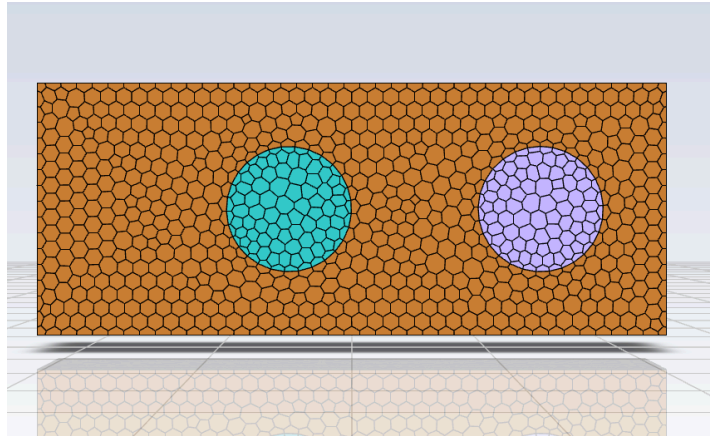
Assuming that a +/-7% error is adequate for the analysis and serves as an anchor, the Ansys parameterization tool is used to determine the heat transfer coefficient for air velocities ranging from 0.01 to 250 m/s for each model.

Cylinders in Series, Parallel, and Matrix Configuration

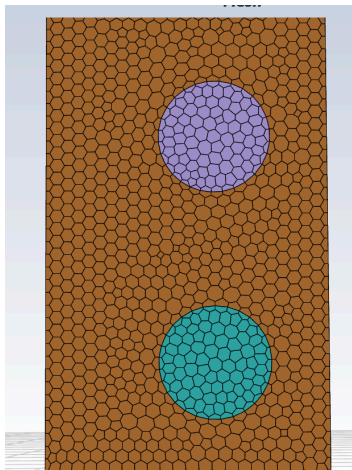
Each of the following models use the models, boundary conditions, mesh metrics, and parameterization ranges as the single cylinder model. This section is intended to serve as a visual to ensure each model is

depicted correctly.

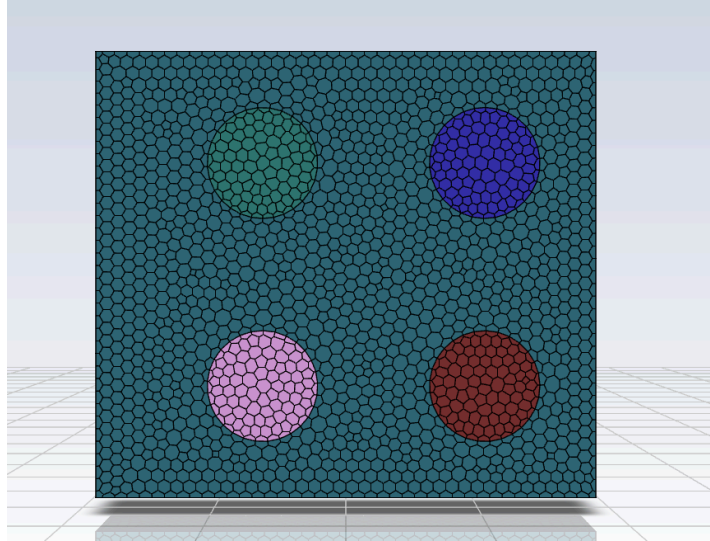
Cylinders in Series



Cylinders in Parallel



Matrix Configuration



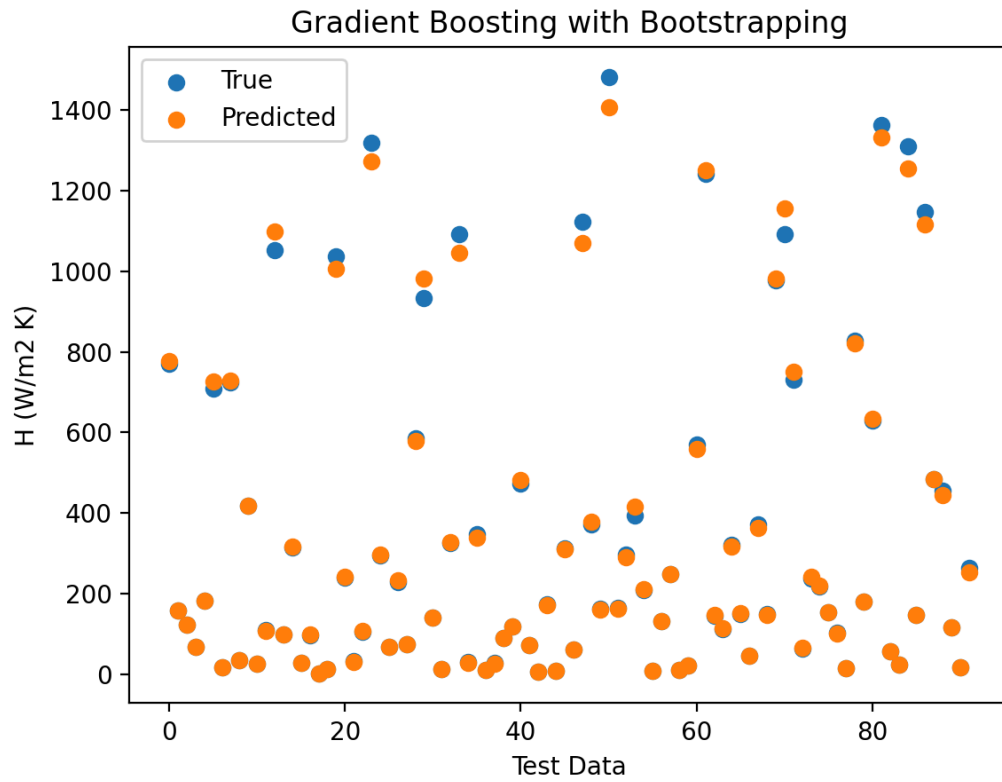
Machine Learning Model

The machine learning model requires a discretization of the model parameters in Ansys Fluent. The features include the following:

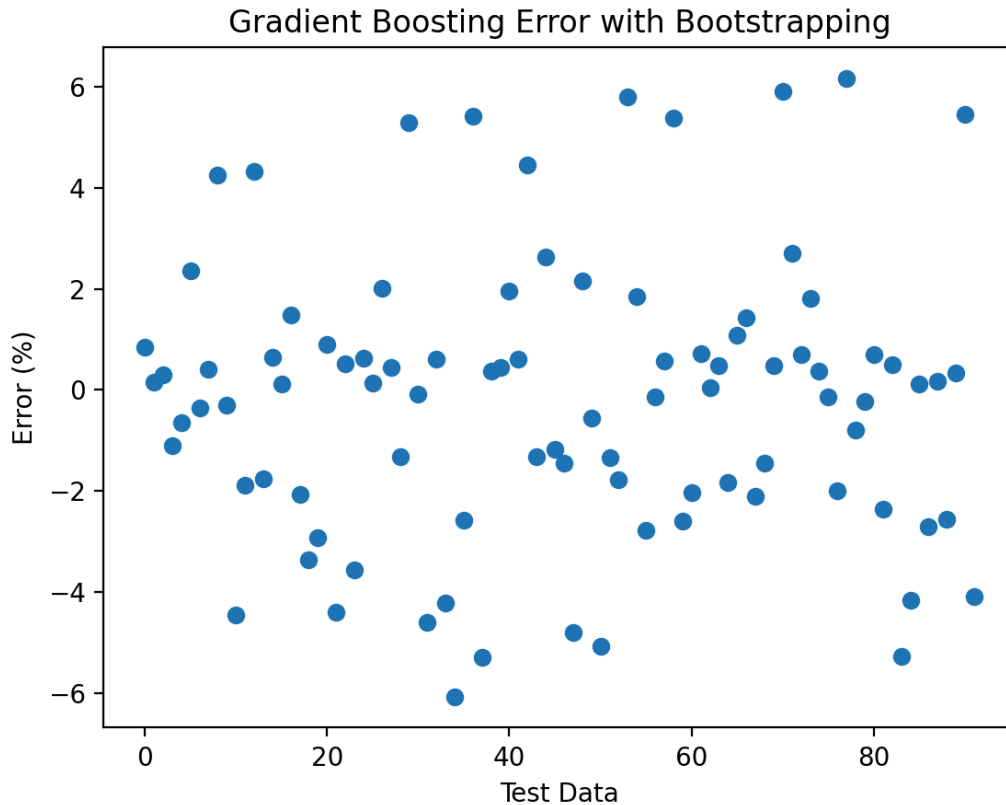
Regressor	Feature 1	Feature 2	Feature 3	Feature 4	Feature 5	Feature 6
Heat Transfer Coefficient	Air Velocity	Cylinder Diameter	# Cylinders	Configuration	# Rows	# Columns

Multiple different Machine Learning models are trained including Linear Regression, Ridge Regression, Decision Trees, and Gradient Boosting. However, the Gradient Boosting model yielded the most accurate results.

The data set, consisting of 456 data points, is split into testing and training data using five-fold cross validation. 80% of the data is used for training and the remaining 20% is used for testing. The Gradient Boosting hyperparameters are tuned using the exhaustive GridSearchCV method. Once tuned, the model uses the Bootstrapping to resample the data and generate results that fit the data better. The testing data is plotted below alongside the predicted data.



To aid in interpreting the results above, the error of the predicted data are plotted in the figure below.



The figure above shows that the Machine Learning model has between a $\pm 6\%$ error when predicting the heat transfer correlation given the specific features used to train the model. The corresponding R^2 Score and Mean Absolute Error are 0.9979 and 8.45, respectively. Similar plots are generated for the Ridge Regression model in the appendix to show the impact that the Machine Learning model has on the accuracy.

Summary

In short, the single cylinder heat transfer correlation model in Ansys Fluent is anchored using first principles with an error less than $\pm 7\%$. Then, the model is parameterized and ran with 114 air velocities for the single, two in series, two in parallel, and matrix of four cylinder configurations. The data consisting of 456 datapoints is then split into testing and training data sets to train multiple machine learning models. Gradient Boosting yields the best results with an error of $\pm 6\%$ with the testing data, slightly less than the error of the CFD results.

The results indicate simulations can be parameterized to create surrogate models that use less resources to predict outcomes of simulations than computationally intensive programs such as Ansys Fluent.

Works Cited

[1] Bergman, T. L. (2011). *Fundamentals of heat and mass transfer*. John Wiley & Sons.

Appendix

First-Principles Heat Transfer Correlation

```

#Calculates the heat transfer coefficient for single cylinder in cross flow

#Function to calculate Reynolds number
def Reynolds(rho, v, D, mu):
    return rho*v*D/mu

#Function to calculate the heat transfer coefficient of cylinder in cross flow
def h_value(Re, k, D, Pr):
    if Re>0.4 and Re<4:
        C = 0.989
        m = 0.330

    elif Re>= 4 and Re<40:
        C = 0.911
        m = 0.385

    elif Re>=40 and Re<4000:
        C = 0.683
        m = 0.466

    elif Re>=4000 and Re<40000:
        C = 0.193
        m = 0.618

    elif Re>=40000 and Re<400000:
        C = 0.027
        m = 0.805

    return (k/D)*C*(Re**(m))*(Pr**(1/3))

#Properties of Air at 27 C, 300 k
rho, mu, k, Pr = 1.1614, 184.6e-7, 26.3e-3, 0.707 #kg/m3, Ns/m2, W/mk, unitless

#Characteristic Diameter of Cylinder in Cross Flow
D = 0.0254 #m

#Varying Air Velocities for Cross Flow
velocities = [3,5,7.5]

#Creating a loop to calculate the heat transfer coefficient for each air velocity
for v in velocities:

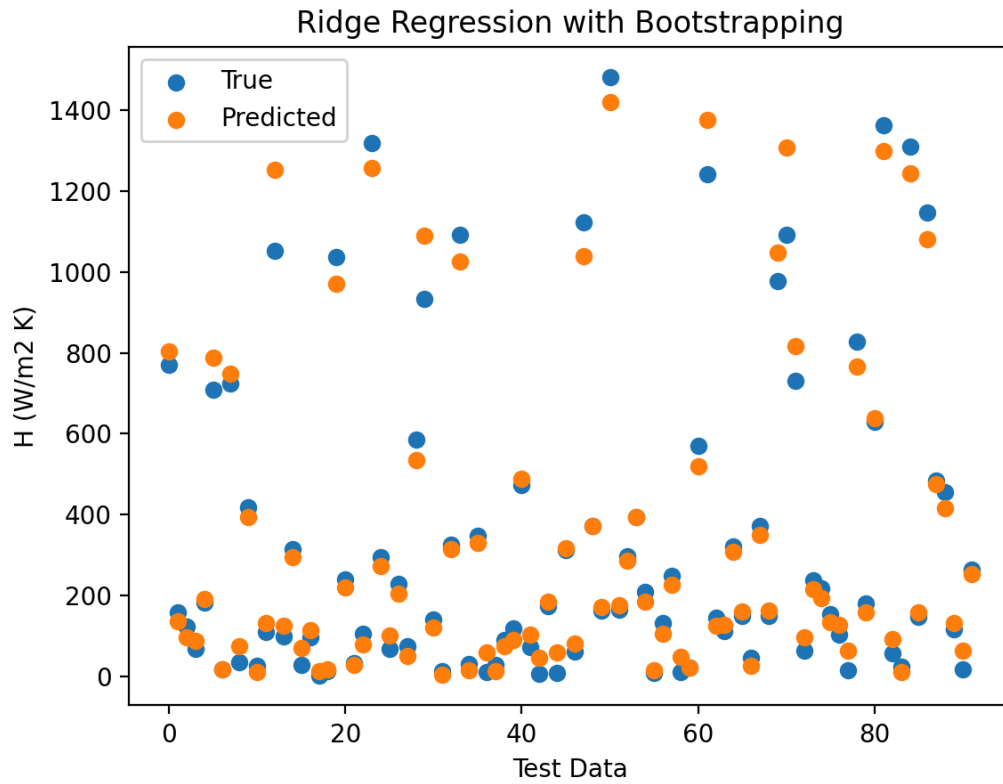
    #Calculating Reynolds Number
    Re = Reynolds(rho, v, D, mu)
    print("Reynolds")
    print(Re)

```

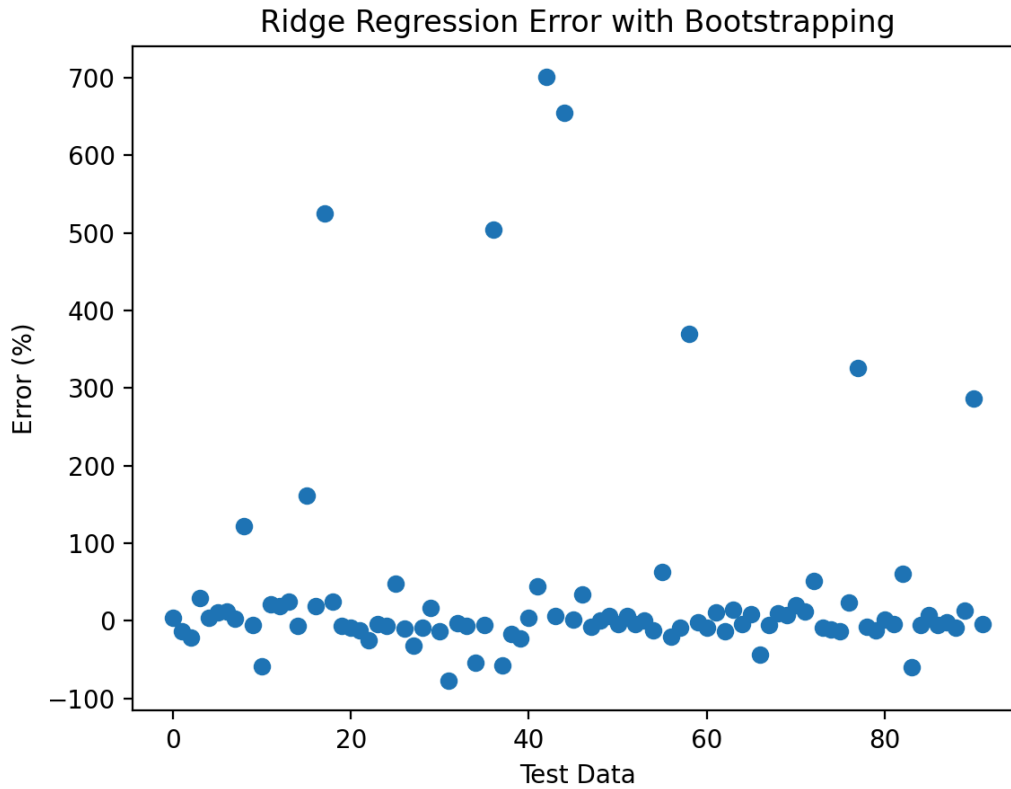


```
#Using h_value function to calculate heat transfer coefficient
print("H-value")
print(h_value(Re, k, D, Pr))
```

Ridge Regression Results



Ridge Regression Error



*The corresponding R^2 Score and Mean Absolute Error are 0.9832 and 34.31, respectively.

Ridge Regression Script

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Ridge
from sklearn.utils import resample
from sklearn.metrics import r2_score, mean_absolute_error
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

#Importing the Data from excel file
Data = np.array(pd.read_excel(r'C:\Users\Trevo\Documents\CFD_Surrogate_Data.xlsx'))

#Categorizing the data into the regressor (y) and the features (x)
y = Data[:,0]
X = Data[:,1:]

#Splitting the data into the testing and training sets
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size =0.2,random_state=0)

#Creating a range for the applicable hyperparameters needed for hypertuning
```

```

parameters = {"alpha": np.arange(0.1, 10, 0.1)}

#Creating the Ridge Regressor
regressor = Ridge()

#Using GridSearchCV for an exhaustive search through all hyperparameters
tuning_model = GridSearchCV(regressor, param_grid = parameters, scoring = 'neg_mean_squared_error', c
v=5,verbose=3, n_jobs=-1)
tuning_model.fit(X_train,y_train)

#Extracting the best hyperparameters from the tuning_model
best_model = tuning_model.best_estimator_
# Bootstrapping: Using multiple samples to fit and predict
n_bootstrap_samples =365 # Number of bootstrap samples
y_pred_train_bootstrap=np.zeros((n_bootstrap_samples,len(X_train)))
y_pred_bootstrap = np.zeros((n_bootstrap_samples, len(X_test)))

# Bootstrap sampling and fitting models
for i in range(n_bootstrap_samples):
    # Generate a bootstrap sample
    X_train_bootstrap, y_train_bootstrap = resample(X_train, y_train, random_state=i)

    # Fit the model using the best hyperparameters to the bootstrap sample
    best_model.fit(X_train_bootstrap, y_train_bootstrap)

    # Predict on the test set and store the results
    y_pred_train_bootstrap[i, :] = best_model.predict(X_train)
    y_pred_bootstrap[i, :] = best_model.predict(X_test)

# Averaging the predictions from all bootstrap models
y_pred_train = np.mean(y_pred_train_bootstrap,axis=0)
y_pred = np.mean(y_pred_bootstrap, axis=0)

# Calculating the relative percentage error
percentage_error = []
for i in range(0, len(y_pred)):
    if y_test[i] != 0: # Avoid division by zero
        percentage_error.append(100 * (y_pred[i] - y_test[i]) / y_test[i])
    else:
        percentage_error.append(np.nan) # Set as NaN or handle as needed

# Calculate the accuracy metrics
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)

# Print the results
print(f'R^2 Score: {r2}')
print(f'Mean Absolute Error: {mae}')

```

```

#Plotting the True and Predicted regressor for the test data
x = np.arange(0,len(X_test))
plt.figure(1)
plt.scatter(x, np.transpose(y_test), label = "True")
plt.scatter(x, np.transpose(y_pred), label="Predicted")
plt.legend()
plt.ylabel('H (W/m2 K)')
plt.xlabel('Test Data')
plt.title('Ridge Regression with Bootstrapping')

#Plotting the accuracy of the predicted regressors
plt.figure(2)
plt.scatter(x,percentage_error)
plt.xlabel('Test Data')
plt.ylabel('Error (%)')
plt.title('Ridge Regression Error with Bootstrapping')
plt.show()

```

Gradient Boosting Script

```

from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.utils import resample
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import r2_score, mean_absolute_error

# Importing the Data from excel file
Data = np.array(pd.read_excel(r'C:\Users\Trevo\Documents\CFD_Surrogate_Data.xlsx'))

# Categorizing the data into the regressor (y) and the features (X)
y = Data[:, 0]
X = Data[:, 1:5]

# Splitting the data into the testing and training sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Creating a range for the applicable hyperparameters needed for hypertuning
parameters = {
    'n_estimators': np.arange(250, 400, 1),
    'learning_rate': np.arange(0.2, 0.3, 0.01),
    'max_depth': np.arange(1, 12,1)
}

# Creating the Gradient Boosting Regressor

```

```

regressor = GradientBoostingRegressor()

# Using GridSearchCV for an exhaustive search through all hyperparameters
tuning_model = GridSearchCV(regressor, param_grid=parameters, scoring='neg_mean_squared_error', cv
=5, verbose=3, n_jobs=-1)
tuning_model.fit(X_train, y_train)

# Extracting the best hyperparameters from the tuning_model
best_model = tuning_model.best_estimator_
print(best_model)
# Bootstrapping: Using multiple samples to fit and predict
n_bootstrap_samples = 365 # Number of bootstrap samples
y_pred_train_bootstrap = np.zeros((n_bootstrap_samples, len(X_train)))
y_pred_bootstrap = np.zeros((n_bootstrap_samples, len(X_test)))

# Bootstrap sampling and fitting models
for i in range(n_bootstrap_samples):
    # Generate a bootstrap sample
    X_train_bootstrap, y_train_bootstrap = resample(X_train, y_train, random_state=i)

    # Fit the model using the best hyperparameters to the bootstrap sample
    best_model.fit(X_train_bootstrap, y_train_bootstrap)

    # Predict on the test set and store the results
    y_pred_train_bootstrap[i, :] = best_model.predict(X_train)
    y_pred_bootstrap[i, :] = best_model.predict(X_test)

# Averaging the predictions from all bootstrap models
y_pred_train = np.mean(y_pred_train_bootstrap, axis=0)
y_pred = np.mean(y_pred_bootstrap, axis=0)

# Calculating the relative percentage error
percentage_error = []
for i in range(0, len(y_pred)):
    if y_test[i] != 0: #Avoid division by zero
        percentage_error.append(100 * (y_pred[i] - y_test[i]) / y_test[i])
    else:
        percentage_error.append(np.nan) # Set as NaN or handle as needed

# Calculate the accuracy metrics
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)

# Print the results
print(f'R^2 Score: {r2}')
print(f'Mean Absolute Error: {mae}')

# Plotting the True and Predicted regressor for the test data

```

```

x = np.arange(0, len(X_test))
plt.figure(1)
plt.scatter(x, np.transpose(y_test), label="True")
plt.scatter(x, np.transpose(y_pred), label="Predicted")
plt.legend()
plt.ylabel('H (W/m2 K)')
plt.xlabel('Test Data')
plt.title('Gradient Boosting with Bootstrapping')

# Plotting the accuracy of the predicted regressors
plt.figure(2)
plt.scatter(x, percentage_error)
plt.xlabel('Test Data')
plt.ylabel('Error (%)')
plt.title('Gradient Boosting Error with Bootstrapping')

plt.show()

```